

Wrappers, Decorators, Shims & Dispatch

Four ways to put behavior **around** or **route** behavior to existing code without rewriting it — the machinery behind `@decorators`, compatibility layers, and message routing.

SUBJECT hathor-core · Part I · Track A (programming concepts)

CHAPTER 04 · Foundations · Concepts

BRANCH study/hathor-core-studies

EDITION 2026 · v1

Wrappers · Closures · Decorators · `*args/**kwargs` · `functools.wraps` · Shims · Fallbacks · Dispatch tables · Single dispatch

Table of Contents

Chapter 4 — Wrappers, Decorators, Shims & Dispatch	3
4.1 The unifying idea: behavior around behavior	3
4.2 Wrappers and closures	4
4.3 Decorators — the @ is just sugar	5
4.4 Shims and fallbacks	6
4.5 Dispatch — routing a call to the right code	7
4.6 Bridge — these in hathor-core	8
Recap	9

Chapter 4 — Wrappers, Decorators, Shims & Dispatch

What you'll learn in this chapter

- The single idea behind all four words in the title: adding or routing behavior around existing code without editing that code.
- **Wrappers** and the **closures** that make them work.
- **Decorators** — Python's `@` syntax for wrapping a function — built from scratch, including `*args/**kwargs`, `functools.wraps`, and decorators that take arguments.
- **Shims** and **fallbacks** — small compatibility layers that bridge gaps between versions, platforms, or libraries.
- **Dispatch** — how a program routes a call to the right implementation, from polymorphism to dispatch tables to `singledispatch`.
- A **bridge** to where each appears in `hathor-core` (`@property`, `@inlineCallbacks`, the RocksDB/crypto wrappers, the reactor shim, message-type dispatch).

This chapter finishes the programming-vocabulary set. Everything in it is a variation on one move: don't change the thing — surround it, or route to it. That move shows up under four names — wrapper, decorator, shim, dispatch — and once you see them as facets of the same idea, a lot of otherwise-cryptic Python (`@something` lines, `try/except ImportError`, dictionaries of functions) becomes readable.

It builds directly on Chapter 2's "functions are first-class values" and "higher-order functions," and it closes loops opened earlier: the `@property` from §1.4 and the wrapper/adaptor/proxy of Chapter 3 all land here as special cases.

4.1 The unifying idea: behavior around behavior

You constantly need to add something to existing code — timing, logging, access checks, caching, retries — or to bridge code that doesn't quite line up, or to route a request to one of several handlers. The clumsy way is to edit the original code in every spot. The professional way is to leave the original alone and interpose new code around it, or to centralize the routing decision.

The four ideas in this chapter are all that move, at different scales:

WRAPPER	surround one object/function, forward the call, add behavior
DECORATOR	Python's @ syntax for wrapping a function (a wrapper with sugar)
SHIM	a small layer that bridges a gap (version, platform, library)
DISPATCH	route a call to the correct implementation among several

Wrappers and decorators add behavior; shims reconcile behavior; dispatch selects behavior. Keep that three-way split in mind and the rest of the chapter organizes itself.

4.2 Wrappers and closures

We met wrappers in Chapter 3: the adapter (changes an interface) and the proxy (controls access) are both objects that enclose another object and forward to it. The idea generalizes beyond objects — you can wrap a function too. Suppose you want to know how long some function takes, without editing the function:

```
import time

def timed(fn):
    # takes a function...
    def wrapper(*args, **kwargs):
        # ...and returns a NEW function around it
        start = time.perf_counter()
        result = fn(*args, **kwargs)  # call the original, unchanged
        elapsed = time.perf_counter() - start
        print(f"{fn.__name__} took {elapsed:.4f}s")
        return result  # hand back whatever the original returned
    return wrapper

def slow_add(a, b):
    time.sleep(0.1)
    return a + b

slow_add = timed(slow_add)  # replace the name with the wrapped version
slow_add(2, 3)             # prints "slow_add took 0.1003s", returns 5
```

Two pieces of machinery are doing the heavy lifting here, and both deserve a name.

***args and **kwargs.** The wrapper doesn't know — and shouldn't care — what arguments the wrapped function takes. `*args`¹ captures any positional arguments as a tuple, and `**kwargs`² captures any keyword arguments as a dict; passing them on as `fn(*args, **kwargs)` forwards them faithfully. This is how one wrapper works for any function signature.

Closures. Notice that `wrapper` refers to `fn`, a variable from the enclosing `timed` function — and it still works after `timed` has returned. A function that "remembers" variables from the scope where it was defined is called a **closure**³. The inner `wrapper` closes over `fn`, carrying it along for the rest of its life. Closures are what make

wrappers possible: the returned function keeps a private handle on the thing it wraps. (Closures are also, quietly, how the lambdas-as-callbacks in Chapter 2 captured their surrounding variables.)

The pattern `x = wrap(x)` — replace a name with a wrapped version of itself — is so common that Python gives it dedicated syntax. That syntax is the decorator.

4.3 Decorators — the `@` is just sugar

A **decorator**⁴ is a higher-order function that takes a function and returns a replacement (usually a wrapper around it), applied with the `@` symbol on the line above a definition. The `@` is pure **syntactic sugar**⁵ — a nicer spelling of something you could already write by hand. These two snippets are identical:

```
@timed                                # the decorator form
def slow_add(a, b):
    time.sleep(0.1)
    return a + b

# ...is exactly the same as:

def slow_add(a, b):
    time.sleep(0.1)
    return a + b
slow_add = timed(slow_add)  # the manual form from §4.2
```

That equivalence is the whole secret of decorators. `@timed` above `def slow_add` means "after defining `slow_add`, pass it through `timed` and rebind the name to the result." Nothing more mysterious than §4.2, with tidier syntax.

One refinement you'll always see: `functools.wraps`. When you wrap a function, the wrapper replaces it, so the original's name and docstring get lost — `slow_add.__name__` would become `"wrapper"`, which wrecks logs and debugging. `functools.wraps` is a tiny decorator you apply to your wrapper to copy that metadata across:

```
import functools

def timed(fn):
    @functools.wraps(fn)                # copy fn's name, docstring, etc. onto wrapper
    def wrapper(*args, **kwargs):
        ...
        return fn(*args, **kwargs)
    return wrapper
```

Treat `@functools.wraps(fn)` as boilerplate that means "make the wrapper still look like the original." You'll see it on nearly every well-written decorator.

Decorators that take arguments. Sometimes you want to configure the decorator itself, e.g. `@retry(times=3)`. That needs one more layer: a function that takes the arguments and returns a decorator. Read it as three nested steps — take the config, take the function, return the wrapper:

```
def retry(times):
    def decorator(fn):
        @functools.wraps(fn)
        def wrapper(*args, **kwargs):
            for attempt in range(times):
                try:
                    return fn(*args, **kwargs)
                except Exception:
                    if attempt == times - 1:
                        raise
            return wrapper
        return decorator

    # 1. takes config, returns a decorator
    # 2. takes the function
    # 3. the wrapper

@retry(times=3)
def flaky_call():
    ...

# retry() returns the actual decorator
```

You have already met decorators on classes. `@property` (§1.4) turns a method into a smart attribute; `@staticmethod`⁶ and `@classmethod`⁷ change what a method receives in place of `self`; `@abstractmethod` (§1.7) marks a method as required by an ABC. They are all just decorators — functions that transform the thing defined below them. Recognizing the `@` line as "transform the next definition" is the takeaway.

4.4 Shims and fallbacks

A **shim**⁸ is a small piece of code that sits transparently between two parts of a system to bridge a gap — most often a difference between versions, platforms, or libraries — so that the code on either side can stay unaware of the difference. The word comes from the thin wedge a carpenter slips in to make two mismatched parts fit.

The most common shim you'll see is the **import fallback**: try to use a fast or preferred library; if it isn't available, quietly substitute an alternative that offers the same interface, so the rest of the program never has to know which one it got.

```
try:
    import ujson as json
except ImportError:
    import json

# prefer the fast JSON library...
# ...but fall back to the standard one if absent

# the rest of the file just uses json.loads(...) — oblivious to which it is
```

That `except` -branch is a **fallback**⁹: a safe alternative taken when the preferred path is unavailable or fails. Fallbacks are everywhere in dependable software — a default value when a lookup misses, a backup server when the primary is down, a slower-but-always-present implementation when an optimized one can't load.

Shim vs. adapter. They look similar — both interpose code — but the intent differs (a distinction worth holding). An **adapter** (§3.5) exists to reconcile a genuine interface mismatch by design, and is usually permanent. A **shim** exists to paper over an environmental difference — an older runtime, a missing feature, a platform quirk — and is often temporary, meant to be removed once the gap closes. A close cousin from the web world is the **polyfill**¹⁰: code that supplies a missing feature so older environments behave like newer ones. All three are wrappers; what separates them is why they exist.

4.5 Dispatch — routing a call to the right code

Dispatch¹¹ is the act of deciding which piece of code runs for a given call. Every program dispatches constantly; the question is whether the decision is made for you or by you.

Dynamic dispatch — the kind you already use. When you write `vertex.verify()` and the right `verify` runs depending on whether `vertex` is a block or a transaction, that selection-by-type is **dynamic dispatch**¹² — and it is exactly the polymorphism of §1.6. Python makes this decision automatically at call time, based on the object's actual class. Most dispatch in object-oriented code is this invisible kind, and it is the cleanest: add a new type, give it the method, and existing calls route to it with no changes.

Manual dispatch — the dispatch table. Sometimes the thing you're routing on isn't an object's type but a value: a message's `"type"` field, a command name typed at a terminal, an event name. The naive form is a long `if/elif` ladder; the clean form is a **dispatch table**¹³ — a dictionary mapping each key to the function that handles it:

```
def handle_ping(msg): ...
def handle_block(msg): ...
def handle_tx(msg): ...

HANDLERS = {                                # the dispatch table
    "ping": handle_ping,
    "block": handle_block,
    "tx": handle_tx,
}

def dispatch(msg):
    handler = HANDLERS.get(msg["type"])
    if handler is None:
        raise ValueError(f"no handler for {msg['type']}")
    handler(msg)                             # route to the right function
```

This is the same registry idea as the factory of §3.2, and the same shape as the event loop of §2.4 (which dispatches each event to its callback). Adding a message type is one new dictionary entry — no editing of `dispatch` itself. A network protocol that receives many message kinds is, at its core, one big dispatch table.

Single dispatch by argument type. Python's `functools.singledispatch`¹⁴ is a built-in that picks an implementation based on the type of the first argument — useful when you can't (or don't want to) put the behavior as a method on each type:

```
from functools import singledispatch

@singledispatch
def describe(item):
    return f"something: {item!r}"

@describe.register
def _(item: int):
    return f"the number {item}"

@describe.register
def _(item: list):
    return f"a list of {len(item)} items"
```

It is dispatch (selection of code by type) implemented with decorators (§4.3) and a registry (a table) — a neat convergence of this whole chapter's ideas.

4.6 Bridge — these in `hathor-core`

Every facet above is woven through the codebase. Forward-pointers only; full treatment in the chapters named.

Bridge — wrappers/decorators/shims/dispatch in the codebase:

- **Decorators on the model.** `@property`, `@classmethod`, and `@abstractmethod` appear throughout the vertex classes and the storage/index interfaces — §4.3 — **Chapters 25, 27-28.**
- **`@inlineCallbacks` — the async decorator.** Twisted's `@inlineCallbacks` decorator lets code use `yield` to await Deferreds in straight-line style (the Twisted forerunner of Chapter 2's `async / await`). You'll see it across the networking code — §4.3 + Ch 2 — **Chapters 16 & 34.**
- **Library wrappers.** `hathor/storage/` wraps the `python-rocksdb` library, `hathor/crypto/` wraps `cryptography`, and `hathor/pycoin/` wraps `pycoin` — each a wrapper presenting a node-shaped interface over a third-party tool — §4.2 — **Chapters 27 & 40.**
- **The reactor shim/adaptor.** `hathor/reactor/` interposes a layer so the node can run on either the Twisted or an asyncio-backed reactor — a shim over the environment's event loop — §4.4 — **Chapter 23.**
- **Import/feature fallbacks.** Optional dependencies (e.g. Sentry error reporting) are guarded so the node runs with or without them — the §4.4 fallback — **Chapter 42.**
- **Dispatch tables.** The CLI routes subcommands through a name→module dictionary (you saw this in Chapter 0's `CliManager`); the P2P protocol routes incoming messages by type to handler methods; `PubSubManager` dispatches each event to its subscribers — §4.5 — **Chapters 21, 30, 34.**
- **Dynamic dispatch.** Verification and consensus call shared methods that resolve to each vertex type's own implementation — §4.5 / §1.6 — **Chapters 31-32.**

Recap

Idea	What it does	Built from	Tell-tale sign
Wrapper	Surrounds code, forwards, adds behavior	closures, <code>*args/**kwargs</code>	an inner function returning <code>fn(...)</code>
Closure	A function remembering its defining scope	nested functions	inner fn uses an outer variable
Decorator	<code>@</code> sugar for wrapping a function	higher-order fn + closure	a <code>@name</code> line above <code>def</code>

Idea	What it does	Built from	Tell-tale sign
<code>functools.wraps</code>	Keeps a wrapper looking like the original	a decorator	<code>@functools.wraps(fn)</code> boilerplate
Shim / fallback	Bridges a version/platform/library gap	<code>try/except</code> , wrappers	<code>try: import fast /</code> <code>except: import slow</code>
Dispatch table	Routes a value to its handler	a dict of functions	<code>HANDLERS[key](msg)</code>
Dynamic dispatch	Routes a call by object type	polymorphism (§1.6)	<code>obj.method()</code> picking by type
<code>singledispatch</code>	Routes by first-argument type	decorators + registry	<code>@fn.register</code>

These four words name one habit: interpose, don't rewrite. Wrappers and decorators add behavior around a function while leaving it untouched; shims and fallbacks reconcile code with an imperfect environment; dispatch chooses the right implementation, whether automatically (by type) or explicitly (by a table). With objects (Ch 1), time and callbacks (Ch 2), the patterns that arrange them (Ch 3), and now the wrapping-and-routing machinery (Ch 4), you can read the mechanics of almost any line in `hathor-core`. One programming-vocabulary chapter remains — **Chapter 5, type hints and static typing** — after which Track B turns from how the code is built to what it is about: blockchains, UTXOs, and the DAG.

1. `*args` in a function signature collects any extra positional arguments into a tuple. In a call, `*seq` unpacks a sequence into separate positional arguments. The name `args` is convention; the `*` is what matters. ↩
2. `**kwargs` collects any extra keyword arguments into a dict (`kw` = keyword). In a call, `**d` unpacks a dict into keyword arguments. Together `*args`, `**kwargs` forward an arbitrary argument list unchanged. ↩
3. A closure is a function that retains access to variables from the scope in which it was defined, even after that scope has finished executing. The inner function "closes over" those variables. ↩
4. A decorator is a callable that takes a function (or class) and returns a replacement, applied with `@name` above a definition. It is the standard Python way to wrap behavior around a function. ↩
5. Syntactic sugar is syntax that makes something easier to write or read but adds no new capability — it could always be expressed the longer way. `@decorator` is sugar for `f = decorator(f)`. ↩
6. `@staticmethod` marks a method that receives neither `self` nor the class — a plain function namespaced inside a class. Use it for helpers that logically belong to the class but need no instance. ↩

7. `@classmethod` marks a method whose first argument is the class itself (`cls`) rather than an instance. Common for alternative constructors (e.g. `Vertex.from_bytes(...)`). ↵
8. A shim is a small compatibility layer that transparently intercepts calls and bridges a difference (between API versions, platforms, or libraries), letting surrounding code stay unaware of the difference. ↵
9. A fallback is an alternative path taken when the preferred one is unavailable or fails — a default value, a backup service, or a simpler implementation. It makes code degrade gracefully instead of breaking. ↵
10. A polyfill is code (the term comes from web development) that implements a feature the current environment lacks, so older runtimes behave like newer ones. A kind of shim for missing features. ↵
11. Dispatch is the selection of which code to execute for a given call. It can be automatic (by object type) or explicit (by looking a key up in a table). ↵
12. Dynamic dispatch selects the implementation at run time based on an object's actual type — the mechanism behind method calls and polymorphism. Contrast static dispatch, decided at compile time by the declared type. ↵
13. A dispatch table is a data structure (usually a dict) mapping keys — message types, command names, event names — to the functions that handle them, replacing a long `if/elif` chain. ↵
14. `functools.singledispatch` is a Python standard-library decorator that turns a function into one whose implementation is chosen by the type of its first argument, with implementations registered via `@fn.register` . ↵